Day 16 — Backend Development Introduction

Spring Boot • REST APIs • Layered Architecture • React Integration

1. Core Concepts

Backend development is the server-side part of an application. It handles business logic, talks to databases, enforces security, and exposes data to clients (React, mobile apps) through APIs.

Client-server communication: the client sends a request, the server processes it and returns a response.

```
React (browser) --HTTP request--> Spring Boot server --> Database
React (browser) <--HTTP response (JSON)-- Spring Boot server <--
```

HTTP Methods

Method	Purpose	Example
GET	Read data	GET /api/students
POST	Create data	POST /api/students
PUT	Replace/update fully	PUT /api/students/1
PATCH	Update partially	PATCH /api/students/1
DELETE	Remove data	DELETE /api/students/1

HTTP Status Codes

Code	Meaning
200 OK	Success
201 Created	Resource created (POST)
204 No Content	Success, nothing to return (DELETE)
400 Bad Request	Invalid input
401 / 403	Not authenticated / not allowed
404 Not Found	Resource doesn't exist
500 Internal Server Error	Server bug

REST Principles

- Resources are nouns in URLs (`/students`, not `/getStudents`)
- HTTP methods express the action
- The server is stateless
- Data is exchanged as JSON

Request and Response

- **Request:** method, URL, headers, optional body
- **Response:** status code, headers, body

JSON Example

```
{ "id": 1, "name": "Asha", "course": "Java", "active": true }
```

2. Spring Boot and Maven

Spring Boot is a framework that removes most of the configuration work of building Java applications. It provides auto-configuration, an embedded server (Tomcat), and starter dependencies.

Maven is the build tool. Its `pom.xml` file lists dependencies, plugins, and the Java version. Useful commands:

```
mvn clean install     # build
mvn spring-boot:run   # run the app
```

3. Task 1: Set Up the Environment

1. Install a **JDK** (21 is a safe LTS choice; Spring Boot 3+ needs at least 17). Verify with `java -version`.
2. Install an IDE: **IntelliJ IDEA** (Community works) or VS Code with the Java Extension Pack.
3. Maven is bundled with the generated project (`mvnw`), so a separate install is optional.
4. Optional but useful: **Postman** for testing APIs.

4. Task 2: Create the Spring Boot Project

Go to **start.spring.io** and choose:

- Project: **Maven**, Language: **Java**
- Spring Boot: latest stable (not SNAPSHOT)
- Group: `com.example`, Artifact: `demo`
- Packaging: Jar, Java: your installed version
- Dependencies: **Spring Web** (add Spring Data JPA, H2 Database, and Lombok later)

Click **Generate**, unzip, open in your IDE, and run the `DemoApplication` main class. The app starts on `http://localhost:8080`.

Project Structure

```
demo/
|-- pom.xml
`-- src/
    |-- main/
    |   |-- java/com/example/demo/
    |   |   |-- DemoApplication.java   # entry point
    |   |   |-- controller/
    |   |   |-- service/
    |   |   |-- repository/
    |   |   `-- entity/
    |   `-- resources/
    |       `-- application.properties # config (port, DB, etc.)
    `-- test/                          # unit/integration tests
```

5. Task 3: Layered Architecture

```
Controller  ->  Service  ->  Repository  ->  Database
  (HTTP)       (logic)     (data access)    (entities = tables)
```

- **Entity:** a Java class mapped to a database table.
- **Repository:** an interface for database operations (CRUD).
- **Service:** business logic, sitting between controller and repository.
- **Controller:** receives HTTP requests and returns responses.

Minimal example (in-memory list, before adding a database)

entity/Student.java

```
public class Student {
    private Long id;
    private String name;
    // constructor, getters, setters
}
```

service/StudentService.java

```
@Service
public class StudentService {
    private final List<Student> students = new ArrayList<>();

    public List<Student> getAll() { return students; }
    public Student add(Student s) { students.add(s); return s; }
}
```

controller/StudentController.java

```
@RestController
@RequestMapping("/api/students")
public class StudentController {

    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    @GetMapping
    public List<Student> getAll() { return service.getAll(); }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Student add(@RequestBody Student s) { return service.add(s); }
}
```

Run the app and open `http://localhost:8080/api/students`. You should see `[]`. Test the POST in Postman.

Once you add Spring Data JPA, the repository becomes just:

```
public interface StudentRepository extends JpaRepository<Student, Long> {}
```

6. Task 4: How React Talks to the Backend

React runs in the browser and calls your API over HTTP using `fetch` or `axios`:

```
useEffect(() => {
  fetch("http://localhost:8080/api/students")
    .then(res => res.json())
    .then(data => setStudents(data));
}, []);
```

CORS: the browser blocks calls between different origins (React on :3000 or :5173, Spring on :8080) unless the backend allows it. Add this to the controller for now:

```
@CrossOrigin(origins = "http://localhost:5173")
```

Flow: React component → fetch/axios → Spring controller → service → repository → DB → JSON back → React updates state and re-renders.

7. Expected Output Checklist

- [ ] JDK installed, `java -version` works
- [ ] Spring Boot project generated and opens in IDE
- [ ] App runs on port 8080 without errors
- [ ] You can explain controller, service, repository, and entity
- [ ] A working `GET /api/students` endpoint
- [ ] You can describe the React → API → DB flow in your own words

Practice ideas: add a `GET /api/students/{id}` endpoint, return a 404 when not found, and change the port with `server.port=9090` in `application.properties`.